Ejercicio previo 1: lista de órdenes condicional

- Orden utilizada y resultado de las diferentes ejecuciones.

```
user1@base:~$ mkdir /tmp/copia-de-seguridad && { cp /etc/passwd /tmp/copia-de-seguridad/ && ls /tmp/copia-de-
seguridad; }
passwd
user1@base:~$ mkdir /tmp/copia-de-seguridad && { cp /etc/passwd /tmp/copia-de-seguridad/ && ls /tmp/copia-de-
seguridad; }
mkdir: cannot create directory '/tmp/copia-de-seguridad': File exists
```

Ejercicio previo 2: utilizando variables de entorno

- Averiguar e<l contenido de las variables HOME y PS1.

```
user1@base:~$ echo $HOME
/home/user1
user1@base:~$ echo $PS1
\[\e]0;\u@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\03
3[00m\]\$
```

- Cambiar el valor de PS1 para que sea igual a "SO-LAB2_$ "

```
user1@base:~$ PS1="SO-LAB2_$ "
SO-LAB2_$
```

- Responder a la siguiente pregunta:

¿Para qué se usa PS1?

PS1 es la variable que define el prompt principal del intérprete de órdenes. Es decir, determina el texto que aparece en la terminal indicando que el sistema está esperando una orden.

- Ejecutar una nueva shell en la misma terminal:

bash --norc (Para el que se pregunte por qué se usa el parámetro "--norc". Se usa porque, por defecto, en Debian existe el fichero ".bashrc", que es procesado cada vez que se ejecuta "bash". En este fichero se define el valor de PS1 y, por tanto, es necesario evitar que se ejecute ("--norc") para poder ver la diferencia entre exportar y no exportar "PS1"). Comprobar que PS1 tiene ahora un valor diferente (el valor por defecto que usa Bash). Salir de la shell recién creada usando "exit". Comprobar que en la shell original (a la que se acaba de volver) PS1 sigue teniendo el valor asignado ("SO-LAB2_$").

```
SO-LAB2_$ bash --norc
bash-5.2$ echo $PS1
\s-\v\$
bash-5.2$ exit
exit
SO-LAB2_$
```

- Exportar el valor de PS1 y ejecutar una nueva shell en la misma terminal, igual que antes ("bash --norc").Comprobar que PS1 mantiene el valor asignado en la shell original. Salir de la shell recién creada usando "exit".

```
SO-LAB2_$ export PS1
SO-LAB2_$ bash --norc
SO-LAB2_$ echo $PS1
SO-LAB2_$
SO-LAB2_$ exit
exit
SO-LAB2_$
```

- Responder a las siguiente pregunta:
¿Qué diferencia hay entre exportar y no exportar el valor de PS1?
Cuando una variable no se exporta, solo está disponible en la shell actual, mientras que cuando se exporta, pasa a formar parte del entorno y es heredada por las shells hijas o procesos que se ejecuten desde esa shell.

Ejercicio previo 3: aprovechando la sustitución de órdenes
- Línea de órdenes utilizada y copia del fichero "registro.txt".

```
SO-LAB2_$ echo -e "==== INICIO\nNombre del sistema: $(hostname)\nFecha: $(date)\nUsuarios:\n$(who)\n==== FINA
L\n" >> registro.txt
SO-LAB2_$ cat registro.txt
==== INICIO
Nombre del sistema: base
Fecha: Wed Mar  4 08:39:25 PM CET 2026
Usuarios:
user1    tty7         2026-03-04 13:32 (:0)
user1    pts/0        2026-03-04 13:32 (:0)
user1    pts/1        2026-03-04 13:32 (:0)
==== FINAL
```

Ejercicio previo 4: trabajando con expresiones condicionales
- Copia de la orden.

```
SO-LAB2_$ [[ $n -le 80 && -f "$f" && "$f" == "/usr/bin/date" ]] && echo "Todo OK"
Todo OK
SO-LAB2_$
```

- Copia de las pruebas realizadas. (Incluir tanto la orden como el resultado obtenido).
    1) n > 80

```
SO-LAB2_$ [[ $n -le 80 && -f "$f" && "$f" == "/usr/bin/date" ]] && echo "Todo OK"
SO-LAB2_$
```

    2) fichero distinto

```
SO-LAB2_$ n=50
f="/etc/passwd"
[[ $n -le 80 && -f "$f" && "$f" == "/usr/bin/date" ]] && echo "Todo OK"
SO-LAB2_$
```

Ejercicio previo 5: usando read
- Copia de la orden y una demostración de su funcionamiento.

```
SO-LAB2_$ echo -n "Introduce nombre y edad: "; read nombre edad; echo "$nombre tiene $edad años"
Introduce nombre y edad: Hector 20
Hector tiene 20 años
```